

Module 6

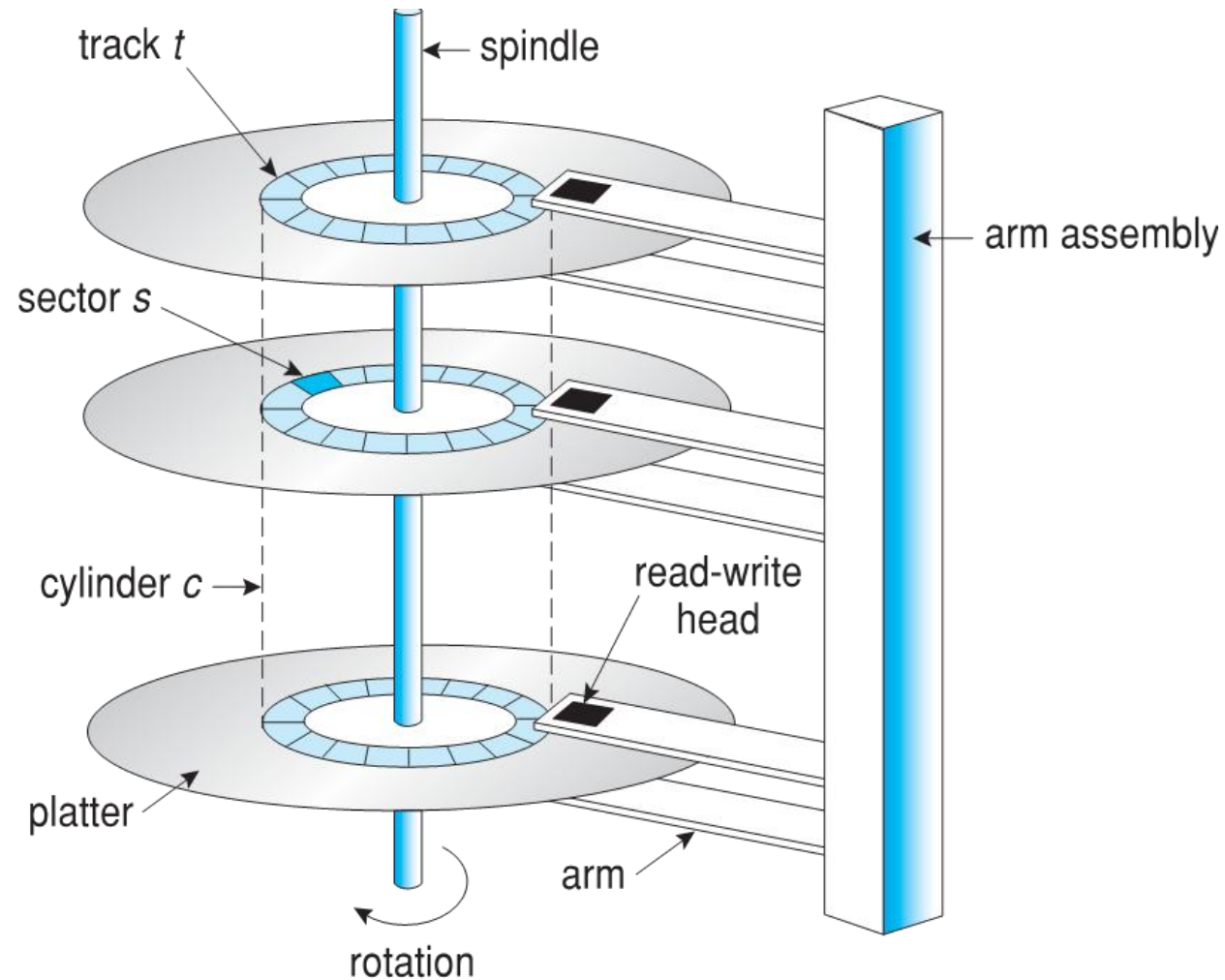
Storage management and security

Dr. Ngangbam Indrason
Assistant Professor - SCOPE
VIT-AP University, Amaravati

Overview of Mass Storage Structure

- **Magnetic disks** provide bulk of secondary storage of modern computers
 - Drives rotate at 60 to 250 times per second
 - **Transfer rate** is rate at which data flow between drive and computer
 - **Positioning time** (**random-access time**) is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**)
 - **Head crash** results from disk head making contact with the disk surface -- That's bad
- Disks can be removable
- Drive attached to computer via **I/O bus**

Moving-head Disk Mechanism



Hard Disks

- Platters range from .85" to 14" (historically)
 - Commonly 3.5", 2.5", and 1.8"
- Range from 30GB to 3TB per drive
- Performance
 - Transfer Rate – theoretical – 6 Gb/sec
 - Effective Transfer Rate – real – 1Gb/sec
 - Seek time from 3ms to 12ms – 9ms common for desktop drives
 - Average seek time measured or calculated based on 1/3 of tracks
 - Latency based on spindle speed (specifically **Rotational Latency**)
 - Latency = $1 / (\text{RPM} / 60) = 60 / \text{RPM}$ (in seconds) [convert mins to secs of RPM]
 - Average latency = $\frac{1}{2}$ latency [half the time of one full revolution]

The First Commercial Disk Drive



- 1956
- IBM RAMDAC computer included the IBM Model 350 disk storage system
- 5M (7 bit) characters
- 50 x 24" platters
- Access time = < 1 second

Solid-State Disks

- Nonvolatile memory used like a hard drive
 - Many technology variations
- Can be more reliable than HDDs
- More expensive per MB
- Maybe have shorter life span
- Less capacity
- But much faster
- Busses can be too slow → connect directly to PCI for example
- No moving parts, so no seek time or rotational latency

Magnetic Tape

- Was early secondary-storage medium
- Relatively permanent and holds large quantities of data
- Access time slow
- Random access ~1000 times slower than disk
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems
- Kept in spool and wound or rewound past read-write head
- Once data under head, transfer rates comparable to disk
 - 140MB/sec and greater
- 200GB to 1.5TB typical storage

Disk Structure

- Disk drives are addressed as large 1-dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer
- The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially
- Sector 0 is the first sector of the first track on the outermost cylinder
- Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost
- Logical to physical address should be easy
 - Except for bad sectors
 - Non-constant # of sectors per track via constant angular velocity

Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
- Seek time $\sim$ seek distance
- Disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer

Disk Scheduling

- There are many sources of disk I/O request
 - OS
 - System processes
 - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
 - Optimization algorithms only make sense when a queue exists

Disk Scheduling

- Note that drive controllers have small buffers and can manage a queue of I/O requests.
- Several algorithms exist to schedule the servicing of disk I/O requests
- The analysis is true for one or many platters
- We illustrate scheduling algorithms with a request queue (0-199)

98, 183, 37, 122, 14, 124, 65, 67

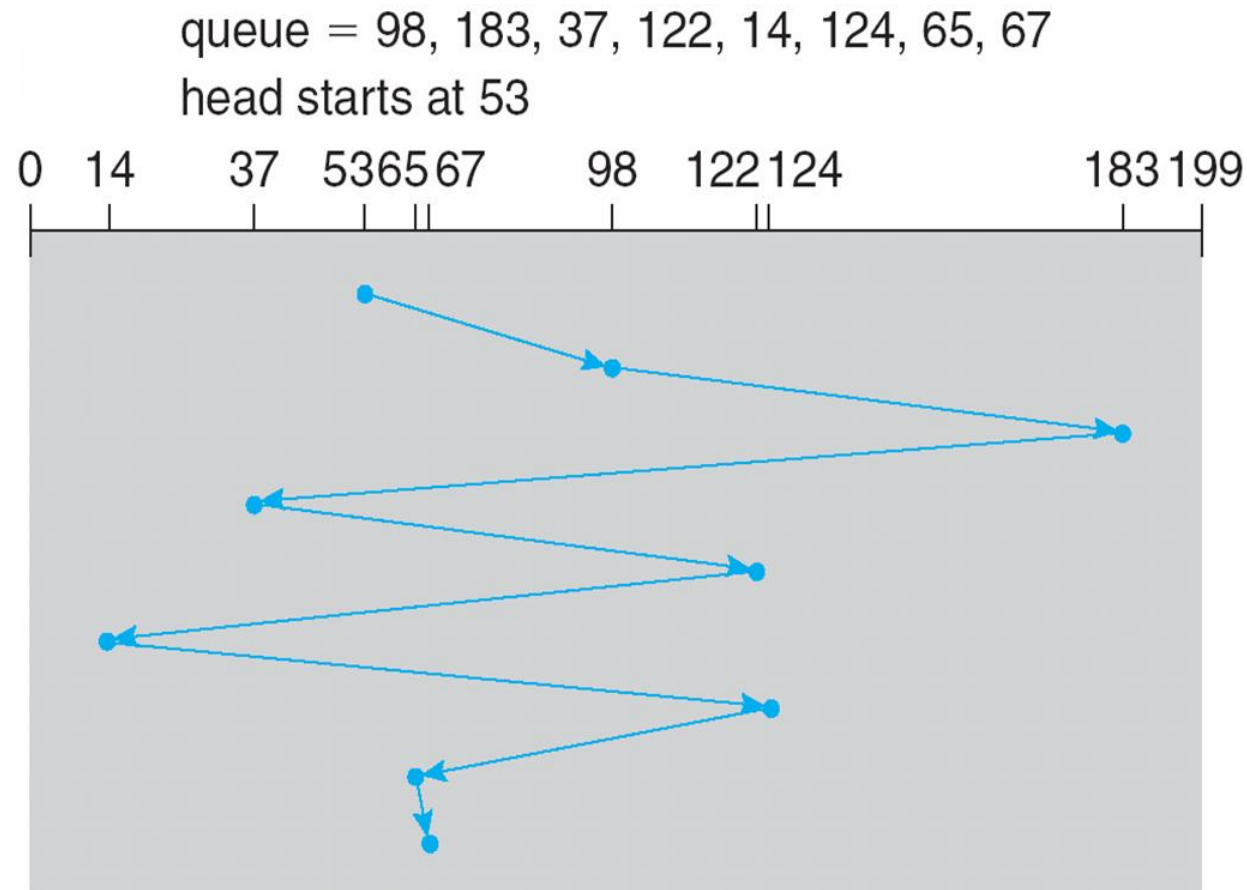
Head pointer 53

First-Come, First-Served (FCFS)

- **Mechanism:** Requests are serviced strictly in the order they arrive in the queue.
- **Performance:** Simple to implement and fair, but typically the least efficient. It does not optimize arm movement, leading to long seeks if requests are scattered across the disk.

First-Come, First-Served (FCFS)

- Illustration shows total head movement of 640 cylinders

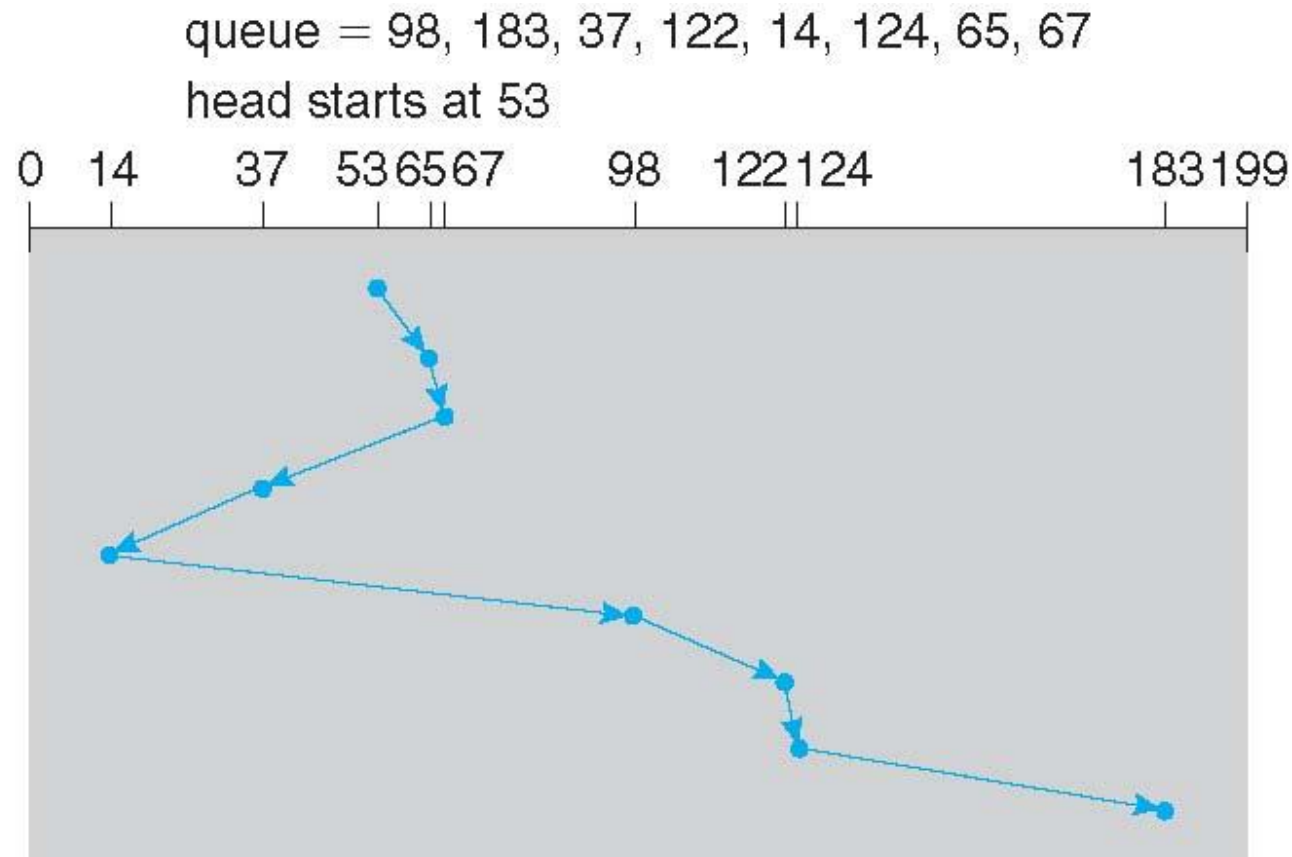


Shortest Seek Time First (SSTF)

- **Mechanism:** Selects the request that is closest to the current head position.
- **Performance:** Significantly improves throughput compared to FCFS by minimizing the total seek distance.
- **Issue:** Can lead to starvation, where requests far from the current head position may never be serviced if a continuous stream of closer requests keeps arriving.

Shortest Seek Time First (SSTF)

- Illustration shows total head movement of 236 cylinders



SCAN (Elevator Algorithm)

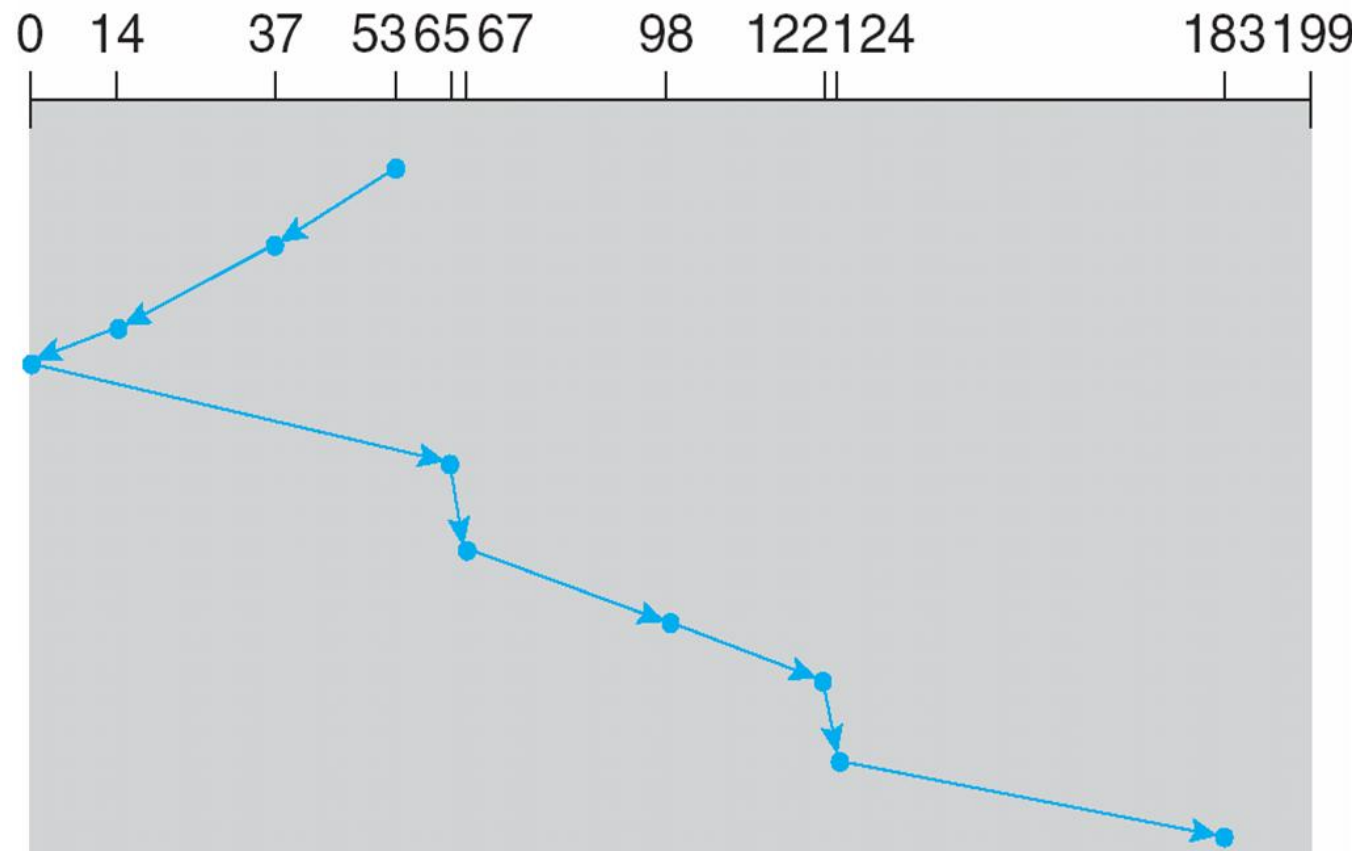
- **Mechanism**: The disk arm starts at one end of the disk and moves toward the other end, servicing all requests in its path. When it reaches the end, it reverses direction and services requests along the way back.
- **Analogy**: Simulates an elevator in a building, picking up and dropping off passengers as it travels up and down.
- **Performance**: Provides a more uniform wait time than SSTF and avoids starvation.
- **Issue**: Requests just serviced by the arm's sweep and those at the far end of the current sweep have to wait the longest.

SCAN

Illustration shows total head movement of 208 cylinders

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



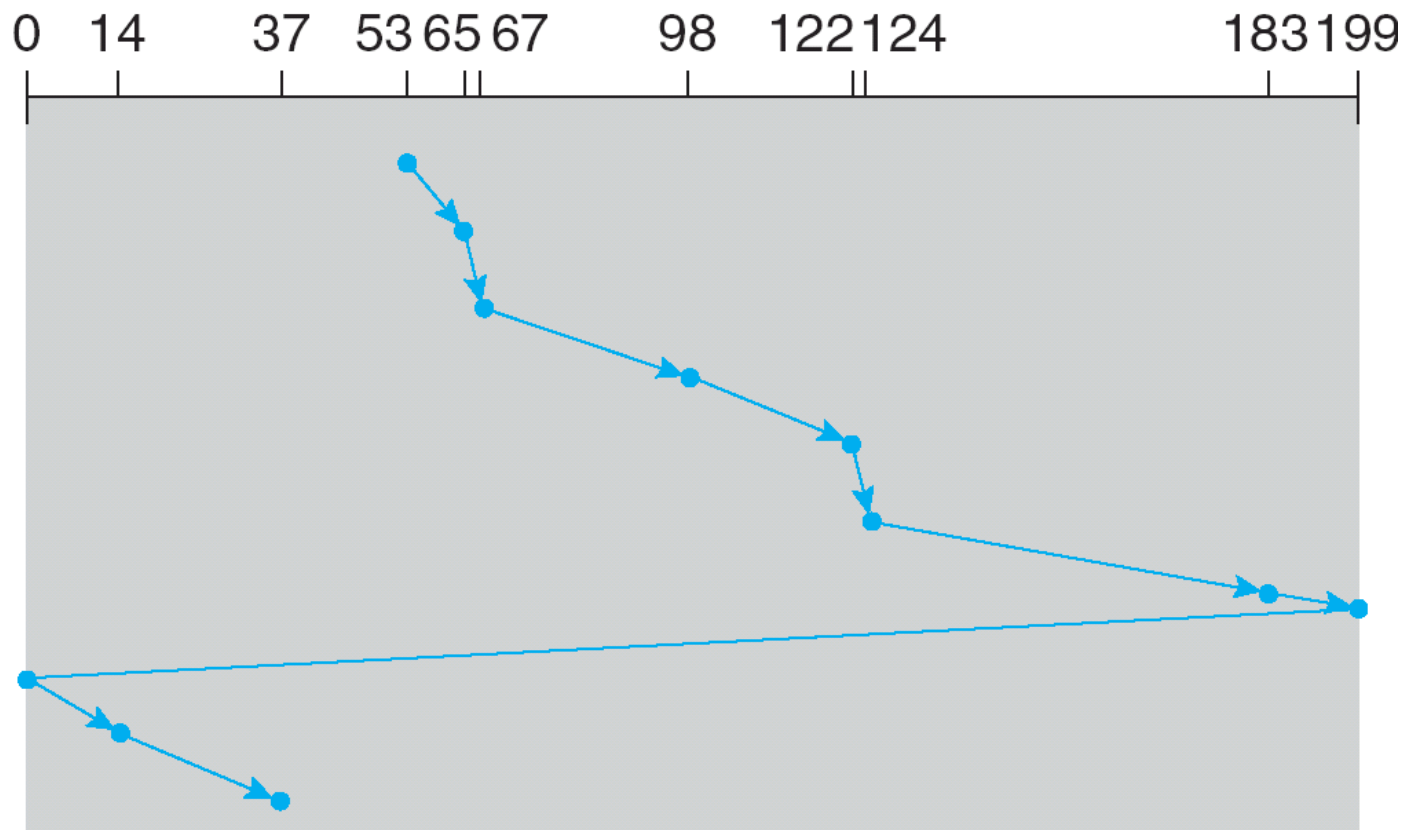
C-SCAN (Circular SCAN)

- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
- When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one
- Total number of cylinders?

C-SCAN

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



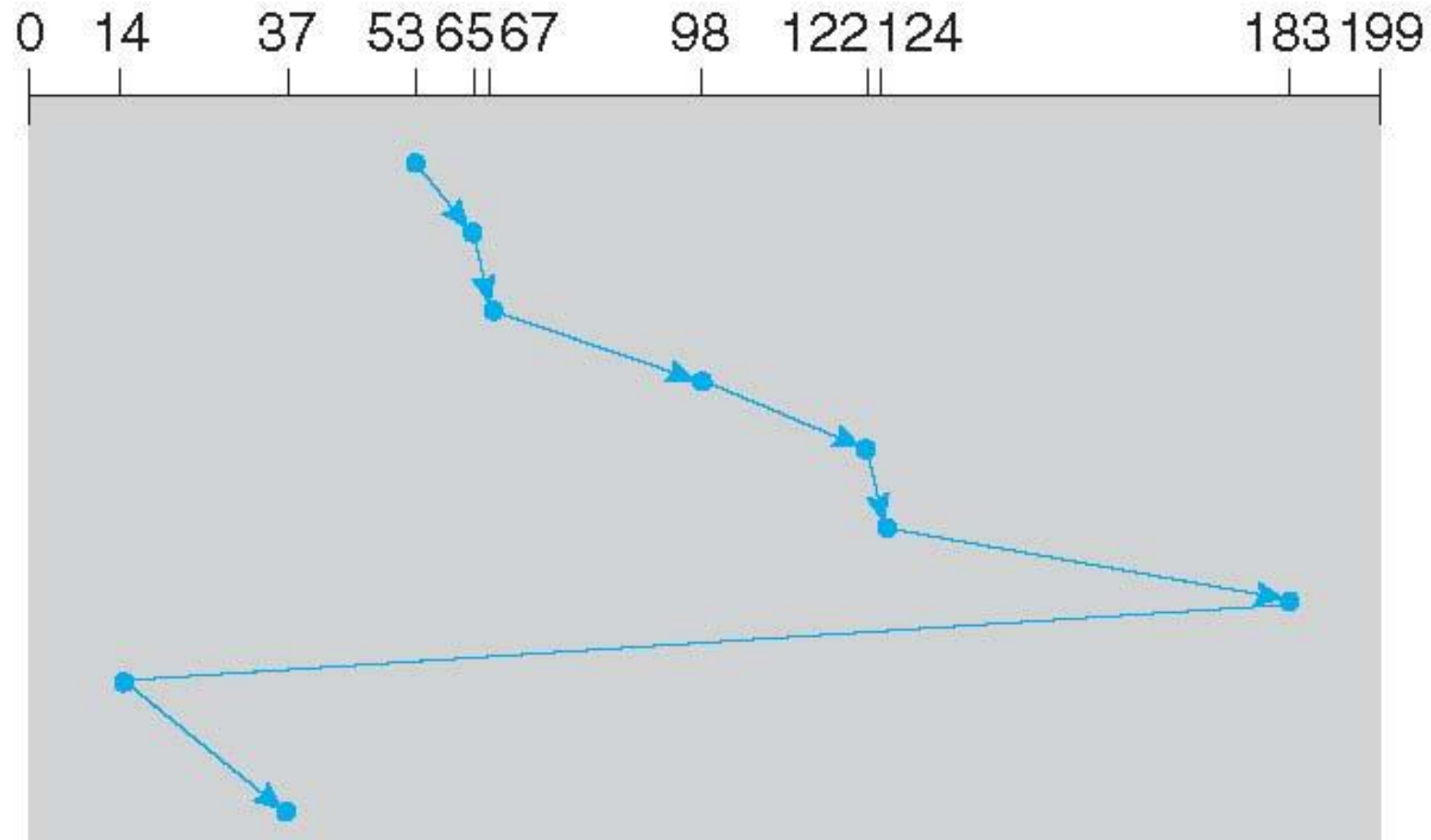
LOOK and C-LOOK

- LOOK a version of SCAN, C-LOOK a version of C-SCAN
- **LOOK** is a refinement of the SCAN algorithm. The arm only travels as far as the last request in a particular direction, and then immediately reverses direction. It does not travel all the way to the physical end of the disk.
- It reduces the total seek time compared to SCAN by eliminating the unnecessary trip to the absolute edge of the disk when no requests are there.
- **C-LOOK** combines the uniformity of C-SCAN with the seek time efficiency of LOOK, making it very effective.
- Total number of cylinders?

C-LOOK

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
 - Less starvation
- Performance depends on the number and types of requests
- Requests for disk service can be influenced by the file-allocation method
- The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for the default algorithm

Protection

- File owner/creator should be able to control:
 - what can be done
 - by whom
- Types of access
 - Read
 - Write
 - Execute
 - Append
 - Delete
 - List

Access Control List (ACL)

- An **Access Control List (ACL)** is a list, associated with a resource (an object like a file, directory, or socket), that explicitly defines which subjects (users, groups, or processes) are granted or denied specific access rights.
- An ACL is structured as a table of ordered entries, where each entry is a (Subject, Permission) pair:

$$ACL_{Object} = \{ (Subject_1, Right_1), (Subject_2, Right_2), \dots \}$$

- **Subject (Who)**: Specifies the identity (e.g., User ID, Group ID, or role).
- **Resource (What)**: The object being protected (e.g., /home/user/private.doc).
- **Permission/Rights (How)**: Defines the allowed action (e.g., Read, Write, Execute, Delete, Append).

Access Control List (ACL)

- Operation
 - When a subject attempts to perform an operation on a resource, the OS checks the resource's ACL.
 - The OS scans the ACL entries in order.
 - If a match is found for the subject, the system checks if the requested operation is permitted by the granted rights.
 - If the requested right is explicitly granted, access is allowed.
 - If no explicit entry is found, or if the rights are denied, access is usually rejected (default deny).

Access Control List (ACL)

- Advantages:
 - **Fine-Grained Control**: ACLs allow for very specific and complex permission rules (e.g., "User X can only read this file, but Group Y can read and write").
 - **Easy Revocation**: To revoke access for a specific user, you only need to modify the single entry associated with that user on the resource.
- Disadvantage:
 - **Management Overhead**: For a large number of users and files, ACLs can become very long and difficult to manage, especially when changing permissions for a specific user across many resources.

ACL Management Commands

Command	Purpose
getfacl	View the current ACLs for a file or directory.
setfacl	Set or modify the ACL entries for a file or directory.

- The *setfacl* command uses the *-m* option to modify or set new ACL entries. The basic format is:

```
$ setfacl -m [entry_type]:[name]:[permissions] [file_or_directory]  
setfacl -m u:alice:rw project_report.txt [user: alice, read write]  
setfacl -m g:devs:rx project_report.txt [group: devs, read execute]
```

ACL Management Commands

- To revoke permissions, you can use two main methods:
 - **Method 1: Remove Specific Entries (-x)**
 - The -x option is used to remove a specific, existing ACL entry for a user or group.

```
$ setfacl -x u:alice project_report.txt
```

- **Method 2: Set Permissions to None (Revoking Rights)**
- You can also revoke rights by using the -m (modify) option and setting the permissions to nothing (---).

```
$ setfacl -m g:devs:--- project_report.txt
```

ACL Management Commands

- The *getfacl* command shows the current access permissions, including the standard Unix permissions and all added ACL entries.

- Example: View the ACL for project_report.txt:

```
$ getfacl project_report.txt
```

```
# file: project_report.txt
```

```
# owner: bob
```

```
# group: staff
```

```
user::rw-
```

```
# Owner (bob) has rw-
```

```
user:alice:rw-
```

```
# Specific ACL entry for alice
```

```
group::r--
```

```
# Owing group (staff) has r--
```

```
group:devs:r-x
```

```
# Specific ACL entry for devs
```

```
mask::rw-
```

```
# Mask (maximum allowed permissions for all ACL entries)
```

```
other::r--
```

```
# Others have r--
```

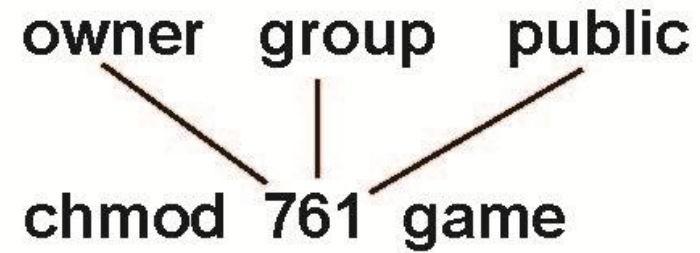
Access Lists and Groups

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

a) owner access	7	→	$\overset{\text{RWX}}{111}$
b) group access	6	→	$\overset{\text{RWX}}{110}$
c) public access	1	→	$\overset{\text{RWX}}{001}$

- Ask manager to create a group (unique name), say **G**, and add some users to the group.
- For a particular file (say **game**) or subdirectory, define an appropriate access.

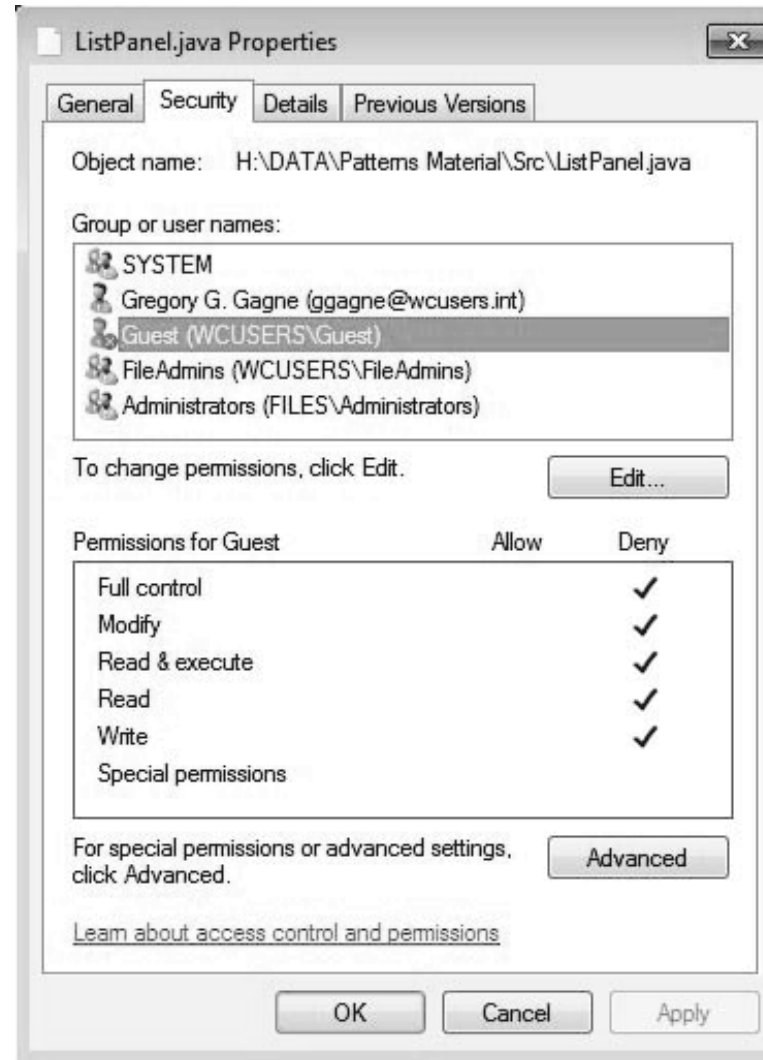
Access Lists and Groups



- Attach a group to a file

`chgrp` `G` `game`

Windows 7 Access-Control List Management



A Sample UNIX Directory Listing

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5 pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx-----	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/

End of Module 6